

Neural Lander: Stable Drone Landing Control Using Learned Dynamics

Guanya Shi^{*,1}, Xichen Shi^{*,1}, Michael O’Connell^{*,1}, Rose Yu², Kamyar Azizzadenesheli³, Animashree Anandkumar¹, Yisong Yue¹, and Soon-Jo Chung¹

Abstract—Precise near-ground trajectory control is difficult for multi-rotor drones, due to the complex aerodynamic effects caused by interactions between multi-rotor airflow and the environment. Conventional control methods often fail to properly account for these complex effects and fall short in accomplishing smooth landing. In this paper, we present a novel deep-learning-based robust nonlinear controller (*Neural-Lander*) that improves control performance of a quadrotor during landing. Our approach combines a nominal dynamics model with a Deep Neural Network (DNN) that learns high-order interactions. We apply spectral normalization (SN) to constrain the Lipschitz constant of the DNN. Leveraging this Lipschitz property, we design a nonlinear feedback linearization controller using the learned model and prove system stability with disturbance rejection. To the best of our knowledge, this is the first DNN-based nonlinear feedback controller with stability guarantees that can utilize arbitrarily large neural nets. Experimental results demonstrate that the proposed controller significantly outperforms a Baseline Nonlinear Tracking Controller in both landing and cross-table trajectory tracking cases. We also empirically show that the DNN generalizes well to unseen data outside the training domain.

I. INTRODUCTION

Unmanned Aerial Vehicles (UAVs) require high precision control of aircraft positions, especially during landing and take-off. This problem is challenging largely due to complex interactions of rotor and wing airflows with the ground. The aerospace community has long identified such ground effect that can cause an increased lift force and a reduced aerodynamic drag. These effects can be both helpful and disruptive in flight stability [1], and the complications are exacerbated with multiple rotors. Therefore, performing automatic landing of UAVs is risk-prone, and requires expensive high-precision sensors as well as carefully designed controllers.

Compensating for ground effect is a long-standing problem in the aerial robotics community. Prior work has largely focused on mathematical modeling (e.g. [2]) as part of system identification (ID). These models are later used to approximate aerodynamics forces during flights close to the ground and combined with controller design for feed-forward cancellation (e.g. [3]). However, existing theoretical ground effect models are derived based on steady-flow conditions, whereas most practical cases exhibit unsteady flow. Alternative approaches, such as integral or adaptive control methods, often suffer from slow response and delayed feedback. [4] employs Bayesian Optimization for open-air control but not for take-off/landing. Given these limitations, the precision of existing

fully automated systems for UAVs are still insufficient for landing and take-off, thereby necessitating the guidance of a human UAV operator during those phases.

To capture complex aerodynamic interactions without overly-constrained by conventional modeling assumptions, we take a machine-learning (ML) approach to build a black-box ground effect model using Deep Neural Networks (DNNs). However, incorporating such models into a UAV controller faces three key challenges. First, it is challenging to collect sufficient real-world training data, as DNNs are notoriously data-hungry. Second, due to high-dimensionality, DNNs can be unstable and generate unpredictable output, which makes the system susceptible to instability in the feedback control loop. Third, DNNs are often difficult to analyze, which makes it difficult to design provably stable DNN-based controllers.

The aforementioned challenges pervade previous works using DNNs to capture high-order non-stationary dynamics. For example, [5], [6] use DNNs to improve system ID of helicopter aerodynamics, but not for controller design. Other approaches aim to generate reference inputs or trajectories from DNNs [7]–[10]. However, these approaches can lead to challenging optimization problems [7], or heavily rely on well-designed closed-loop controller and require a large number of labeled training data [8]–[10]. A more classical approach of using DNNs is direct inverse control [11]–[13] but the non-parametric nature of a DNN controller also makes it challenging to guarantee stability and robustness to noise. [14] proposes a provably stable model-based Reinforcement Learning method based on Lyapunov analysis, but it requires a potentially expensive discretization step and relies on the native Lipschitz constant of the DNN.

Contributions. In this paper, we propose a learning-based controller, *Neural-Lander*, to improve the precision of quadrotor landing with guaranteed stability. Our approach directly learns the ground effect on coupled unsteady aerodynamics and vehicular dynamics. We use deep learning for system ID of residual dynamics and then integrate it with nonlinear feedback linearization control.

We train DNNs with layer-wise spectrally normalized weight matrices. We prove that the resulting controller is globally exponentially stable under bounded learning errors. This is achieved by exploiting the Lipschitz bound of spectrally normalized DNNs. It has earlier been shown that spectral normalization of DNNs leads to good generalization, i.e. stability in a learning-theoretic sense [15]. It is intriguing that spectral normalization simultaneously guarantees stability both in a learning-theoretic and a control-theoretic sense.

* These authors contributed equally to this work. ¹California Institute of Technology, ²Northeastern University, ³University of California, Irvine.

神经着陆器：使用学习到的动力学实现稳定无人机着陆控制

关亚石^{*,1}, 席晨石^{*,1}, 迈克尔·奥康奈尔^{*,1}, 余玫², 卡米阿尔·阿齐扎德内谢利³, 阿尼玛希里·阿南德库马尔¹, 岳一松¹, 郑顺祖¹

Abstract——由于多旋翼飞行器的气流与环境之间的相互作用造成复杂的空气动力学效应，多旋翼无人机在近地轨迹的精确控制方面存在困难。传统的控制方法通常无法适当地考虑这些复杂效应，因此在实现平稳着陆方面表现不足。在本文中，我们提出了一种新颖的基于深度学习的鲁棒非线性控制器（*Neural-Lander*），它能够提高四旋翼无人机着陆时的控制性能。我们的方法将标称动力学模型与能够学习高阶相互作用的深度神经网络（DNN）相结合。我们应用谱归一化（SN）来约束DNN的利普希茨常数。利用这一利普希茨性质，我们使用所学习的模型设计了非线性反馈线性化控制器，并证明了系统在干扰抑制下的稳定性。据我们所知，这是首个能够提供稳定性保证并可以利用任意大规模数据的基于DNN的非线性反馈控制器。

一、引言

无人驾驶航空器（UAV）需要对飞行器位置进行高精度控制，尤其是在着陆和起飞时。这个问题的挑战主要在于旋翼和机翼气流与地面之间的复杂相互作用。航空航天界长期以来已经认识到这种地面效应，它会导致升力增加和空气动力阻力减小。这些效应在飞行稳定性方面既可能有利，也可能产生干扰[1]，而多旋翼的情况下情况更为复杂。因此，无人机的自动着陆风险较高，需要昂贵的高精度传感器以及精心设计的控制器。

补偿地面效应一直是航空机器人社区的一个长期问题。以往的工作主要集中在数学建模（例如 [2]），作为系统辨识（ID）的一部分。这些模型随后用于在接近地面的飞行中近似空气动力学，并结合控制器设计进行前馈抵消（例如 [3]）。然而，现有的理论地面效应模型是基于稳态流条件推导的，而大多数实际情况表现为非稳态流。替代方法，如积分或自适应控制方法，通常存在响应慢和反馈延迟的问题。[4] 使用贝叶斯优化进行开阔空气控制，但不用于起飞/着陆。鉴于这些限制，现有的精度

用于无人机的全自动系统在着陆和起飞方面仍然不足，因此在这些阶段需要人工无人机操作员的指导。

为了在不受传统建模假设过度约束的情况下捕捉复杂的气动相互作用，我们采用机器学习（ML）方法，使用深度神经网络（DNNs）构建一个黑箱地效模型。然而，将此类模型纳入无人机控制器面临三个关键挑战。首先，收集足够的现实世界训练数据具有挑战性，因为DNNs以数据需求高而著称。其次，由于高维性，DNNs可能不稳定并产生不可预测的输出，这使系统在反馈控制回路中容易不稳定。第三，DNNs通常难以分析，这使得设计可证明稳定的基于DNN的控制器变得困难。

上述挑战普遍存在于以前使用深度神经网络（DNN）来捕捉高阶非平稳动态的研究中。例如，[5]、[6] 使用深度神经网络来改进直升机空气动力学的系统辨识，但并非用于控制器设计。其他方法则旨在从深度神经网络生成参考输入或轨迹 [7]–[10]。然而，这些方法可能导致复杂的优化问题 [7]，或者高度依赖精心设计的闭环控制器，并且需要大量标注的训练数据 [8]–[10]。一种更经典的使用深度神经网络的方法是直接逆控制 [11]–[13]，但深度神经网络控制器的非参数特性也使得保证稳定性和对噪声的鲁棒性具有挑战性。[14] 提出了一种基于Lyapunov分析的可证明稳定的基于模型的强化学习方法，但它可能需要代价高昂的离散化步骤，并依赖于神经网络的本征Lipschitz常数。

Contributions. 在本文中，我们提出了一种基于学习的控制器 *Neural-Lander*，以提高四旋翼着陆的精度，同时保证稳定性。我们的方法直接学习耦合非稳态空气动力学和车辆动力学的地面效应。我们使用深度学习进行残差动力学的系统识别，然后将其与非线性反馈线性化控制相结合。

我们使用逐层谱归一化权重矩阵训练深度神经网络（DNN）。我们证明了所得控制器在有界学习误差下是全局指数稳定的。这是通过利用谱归一化 DNN 的李普希茨界实现的。此前已有研究表明，DNN 的谱归一化能够带来良好的泛化能力，即在学习理论意义上的稳定性 [15]。有趣的是，谱归一化同时保证了学习理论和控制理论意义上的稳定性。

* These authors contributed equally to this work. ¹California Institute of Technology, ²Northeastern University, ³University of California, Irvine.

We evaluate *Neural-Lander* on trajectory tracking of quadrotor during take-off, landing and cross-table maneuvers. *Neural-Lander* is able to land a quadrotor much more accurately than a Baseline Nonlinear Tracking Controller with a pre-identified system. In particular, we show that compared to the baseline, *Neural-Lander* can decrease error in z axis from 0.13 m to 0, mitigate x and y drifts by as much as 90%, in the landing case. Meanwhile, *Neural-Lander* can decrease z error from 0.153 m to 0.027 m, in the cross-table trajectory tracking task.¹ We also demonstrate that the learned model can handle temporal dependency, and is an improvement over the steady-state theoretical models.

II. PROBLEM STATEMENT: QUADROTOR LANDING

Given quadrotor states as global position $\mathbf{p} \in \mathbb{R}^3$, velocity $\mathbf{v} \in \mathbb{R}^3$, attitude rotation matrix $R \in \text{SO}(3)$, and body angular velocity $\boldsymbol{\omega} \in \mathbb{R}^3$, we consider the following dynamics:

$$\dot{\mathbf{p}} = \mathbf{v}, \quad m\dot{\mathbf{v}} = m\mathbf{g} + R\mathbf{f}_u + \mathbf{f}_a, \quad (1a)$$

$$\dot{R} = RS(\boldsymbol{\omega}), \quad J\dot{\boldsymbol{\omega}} = J\boldsymbol{\omega} \times \boldsymbol{\omega} + \boldsymbol{\tau}_u + \boldsymbol{\tau}_a, \quad (1b)$$

where m and J are mass and inertia matrix of the system respectively, $S(\cdot)$ is skew-symmetric mapping. $\mathbf{g} = [0, 0, -g]^\top$ is the gravity vector, $\mathbf{f}_u = [0, 0, T]^\top$ and $\boldsymbol{\tau}_u = [\tau_x, \tau_y, \tau_z]^\top$ are the total thrust and body torques from four rotors predicted by a nominal model. We use $\boldsymbol{\eta} = [T, \tau_x, \tau_y, \tau_z]^\top$ to denote the output wrench. Typical quadrotor control input uses squared motor speeds $\mathbf{u} = [n_1^2, n_2^2, n_3^2, n_4^2]^\top$, and is linearly related to the output wrench $\boldsymbol{\eta} = B_0\mathbf{u}$, with

$$B_0 = \begin{bmatrix} c_T & c_T l_{\text{arm}} & c_T & -c_T l_{\text{arm}} \\ 0 & 0 & 0 & 0 \\ -c_T l_{\text{arm}} & c_Q & c_T l_{\text{arm}} & c_Q \\ -c_Q & -c_Q & -c_Q & c_Q \end{bmatrix}, \quad (2)$$

where c_T and c_Q are rotor force and torque coefficients, and l_{arm} denotes the length of rotor arm. The key difficulty of precise landing is the influence of unknown disturbance forces $\mathbf{f}_a = [f_{a,x}, f_{a,y}, f_{a,z}]^\top$ and torques $\boldsymbol{\tau}_a = [\tau_{a,x}, \tau_{a,y}, \tau_{a,z}]^\top$, which originate from complex aerodynamic interactions between the quadrotor and the environment.

Problem Statement: We aim to improve controller accuracy by learning the unknown disturbance forces $\mathbf{f}_a$ and torques $\boldsymbol{\tau}_a$ in (1). As we mainly focus on landing and take-off tasks, the attitude dynamics is limited and the aerodynamic disturbance torque $\boldsymbol{\tau}_a$ is bounded. Thus position dynamics (1a) and $\mathbf{f}_a$ will our primary concern. We first approximate $\mathbf{f}_a$ using a DNN with spectral normalization to guarantee its Lipschitz constant, and then incorporate the DNN in our exponentially-stabilizing controller. Training is done off-line, and the learned dynamics is applied in the on-board controller in real-time to achieve smooth landing and take-off.

III. DYNAMICS LEARNING USING DNN

We learn the unknown disturbance force $\mathbf{f}_a$ using a DNN with Rectified Linear Units (ReLU) activation. In general, DNNs equipped with ReLU converge faster during training, demonstrate more robust behavior with respect to changes in hyperparameters, and have fewer vanishing gradient problems compared to other activation functions such as *sigmoid* [16].

A. ReLU Deep Neural Networks

A ReLU deep neural network represents the functional mapping from the input $\mathbf{x}$ to the output $f(\mathbf{x}, \boldsymbol{\theta})$, parameterized by the DNN weights $\boldsymbol{\theta} = W^1, \dots, W^{L+1}$:

$$f(\mathbf{x}, \boldsymbol{\theta}) = W^{L+1}\phi(W^L(\phi(W^{L-1}(\dots\phi(W^1\mathbf{x})\dots))), \quad (3)$$

where the activation function $\phi(\cdot) = \max(\cdot, 0)$ is called the element-wise ReLU function. ReLU is less computationally expensive than *tanh* and *sigmoid* because it involves simpler mathematical operations. However, deep neural networks are usually trained by first-order gradient based optimization, which is highly sensitive on the curvature of the training objective and can be unstable [17]. To alleviate this issue, we apply the spectral normalization technique [15].

B. Spectral Normalization

Spectral normalization stabilizes DNN training by constraining the Lipschitz constant of the objective function. Spectrally normalized DNNs have also been shown to generalize well [18], which is an indication of stability in machine learning. Mathematically, the Lipschitz constant of a function $\|f\|_{\text{Lip}}$ is defined as the smallest value such that

$$\forall \mathbf{x}, \mathbf{x}' : \|f(\mathbf{x}) - f(\mathbf{x}')\|_2 / \|\mathbf{x} - \mathbf{x}'\|_2 \leq \|f\|_{\text{Lip}}.$$

It is known that the Lipschitz constant of a general differentiable function f is the maximum spectral norm (maximum singular value) of its gradient over its domain $\|f\|_{\text{Lip}} = \sup_{\mathbf{x}} \sigma(\nabla f(\mathbf{x}))$.

The ReLU DNN in (3) is a composition of functions. Thus we can bound the Lipschitz constant of the network by constraining the spectral norm of each layer $g^l(\mathbf{x}) = \phi(W^l\mathbf{x})$. Therefore, for a linear map $g(\mathbf{x}) = W\mathbf{x}$, the spectral norm of each layer is given by $\|g\|_{\text{Lip}} = \sup_{\mathbf{x}} \sigma(\nabla g(\mathbf{x})) = \sup_{\mathbf{x}} \sigma(W) = \sigma(W)$. Using the fact that the Lipschitz norm of ReLU activation function $\phi(\cdot)$ is equal to 1, with the inequality $\|g_1 \circ g_2\|_{\text{Lip}} \leq \|g_1\|_{\text{Lip}} \cdot \|g_2\|_{\text{Lip}}$, we can find the following bound on $\|f\|_{\text{Lip}}$:

$$\|f\|_{\text{Lip}} \leq \|g^{L+1}\|_{\text{Lip}} \cdot \|\phi\|_{\text{Lip}} \cdots \|g^1\|_{\text{Lip}} = \prod_{l=1}^{L+1} \sigma(W^l). \quad (4)$$

In practice, we can apply spectral normalization to the weight matrices in each layer during training as follows:

$$\bar{W} = W / \sigma(W) \cdot \gamma^{\frac{1}{L+1}}, \quad (5)$$

where γ is the intended Lipschitz constant for the DNN. The following lemma bounds the Lipschitz constant of a ReLU DNN with spectral normalization.

Lemma 3.1: For a multi-layer ReLU network $f(\mathbf{x}, \boldsymbol{\theta})$, defined in (3) without an activation function on the output layer. Using spectral normalization, the Lipschitz constant of the entire network satisfies:

$$\|f(\mathbf{x}, \bar{\boldsymbol{\theta}})\|_{\text{Lip}} \leq \gamma,$$

with spectrally-normalized parameters $\bar{\boldsymbol{\theta}} = \bar{W}^1, \dots, \bar{W}^{L+1}$.

Proof: As in (4), the Lipschitz constant can be written as a composition of spectral norms over all layers. The proof follows from the spectral norms constrained as in (5). ■

¹Demo videos: <https://youtu.be/FLsG0S78ik>

我们评估了 *Neural-Lander* 在四旋翼起飞、着陆和跨台机动过程中的轨迹跟踪性能。 *Neural-Lander* 能够比具有预先识别系统的基线非线性跟踪控制器更准确地着陆四旋翼。特别地，我们展示了与基线相比， *Neural-Lander* 能够将 z 轴的误差从 0.13 米降低到 0，在着陆情况下减小 x 和 y 漂移最多达 90%。同时，在跨台轨迹跟踪任务中， *Neural-Lander* 能够将 z 误差从 0.153 米降低到 0.027 米。¹ 我们还证明了所学习的模型能够处理时间依赖性，并且相比稳态理论模型有所改进。

II. 问题陈述：四旋翼着陆

给定四旋翼的状态为全局位置 $\mathbf{p} \in \mathbb{R}^3$ 、速度 $\mathbf{v} \in \mathbb{R}^3$ 、姿态旋转矩阵 $R \in \text{SO}(3)$ 和机体角速度 $\boldsymbol{\omega} \in \mathbb{R}^3$ ，我们考虑以下动力学：

$$\dot{\mathbf{p}} = \mathbf{v}, \quad m\dot{\mathbf{v}} = m\mathbf{g} + R\mathbf{f}_u + \mathbf{f}_a, \quad (1a)$$

$$\dot{R} = RS(\boldsymbol{\omega}), \quad J\dot{\boldsymbol{\omega}} = J\boldsymbol{\omega} \times \boldsymbol{\omega} + \boldsymbol{\tau}_u + \boldsymbol{\tau}_a, \quad (1b)$$

其中 m 和 J 分别是系统的质量矩阵和惯性矩阵， $S(\cdot)$ 是斜对称映射。 $\mathbf{g} = [0, 0, -g]^\top$ 是重力向量， $\mathbf{f}_u = [0, 0, T]^\top$ 和 $\boldsymbol{\tau}_u = [\tau_x, \tau_y, \tau_z]^\top$ 是由名义模型预测的四个旋翼产生的总推力和机体扭矩。我们用 $\boldsymbol{\eta} = [T, \tau_x, \tau_y, \tau_z]^\top$ 表示输出力矩。典型的四旋翼控制输入使用平方的电机转速 $\mathbf{u} = [n_1^2, n_2^2, n_3^2, n_4^2]^\top$ ，并且与输出力矩 $\boldsymbol{\eta} = B_0\mathbf{u}$ 线性相关。

$$B_0 = \begin{bmatrix} c_T & c_T l_{\text{arm}} & c_T l_{\text{arm}} & c_T l_{\text{arm}} \\ 0 & -c_Q & 0 & 0 \\ -c_Q & 0 & -c_Q & 0 \\ -c_Q & 0 & 0 & -c_Q \end{bmatrix}, \quad (2)$$

其中 c_T 和 c_Q 是旋翼力和扭矩系数， l_{arm} 表示旋翼臂长度。精确着陆的关键难点是未知扰动力 $\mathbf{f}_a = [f_{a,x}, f_{a,y}, f_{a,z}]^\top$ 和扭矩 $\boldsymbol{\tau}_a = [\tau_{a,x}, \tau_{a,y}, \tau_{a,z}]^\top$ 的影响，这些扰动力和扭矩来源于四旋翼与环境之间复杂的气动相互作用。

Problem Statement: 我们的目标是通过学习(1)中的未知扰动力 $\mathbf{f}_a$ 和力矩 $\boldsymbol{\tau}_a$ 来提高控制器的精度。由于我们主要关注着陆和起飞任务，姿态动力学受到限制，气动扰动力矩 $\boldsymbol{\tau}_a$ 是有界的。因此，位置动力学 (1a) 和 $\mathbf{f}_a$ 将是我们的主要关注点。我们首先使用具有谱归一化的深度神经网络(DNN)来近似 $\mathbf{f}_a$ ，以保证其利普希茨常数，然后将该 DNN 融入我们的指数稳定控制器中。训练在离线完成，学到的动力学将在机载控制器中实时应用，以实现平稳的着陆和起飞。

III. 使用深度神经网络的动力学学习

我们使用带有修正线性单元 (ReLU) 激活的深度神经网络 (DNN) 来学习未知扰动力 $\mathbf{f}_a$ 。一般来说，配备 ReLU 的 DNN 在训练过程中收敛更快，对于超参数的变化表现出更稳健的行为，并且相比其他激活函数如 *sigmoid* [16]，梯度消失问题更少。

A. ReLU Deep Neural Networks

一个 ReLU 深度神经网络表示从输入 $\mathbf{x}$ 到输出 $f(\mathbf{x}, \boldsymbol{\theta})$ 的函数映射，由 DNN 权重 $\boldsymbol{\theta} = W^1, \dots, W^{L+1}$ 参数化：

$$f(\mathbf{x}, \boldsymbol{\theta}) = W^{L+1} \phi(W^L (\phi(W^{L-1} (\dots \phi(W^1 \mathbf{x}) \dots))), \quad (3)$$

其中激活函数 $\phi(\cdot) = \max(\cdot, 0)$ 被称为逐元素 ReLU 函数。ReLU 比 *tanh* 和 *sigmoid* 的计算开销更低，因为它涉及更简单的数学运算。然而，深度神经网络通常通过一阶梯度优化进行训练，这对训练目标的曲率高度敏感，并且可能不稳定 [17]。为缓解该问题，我们采用了谱归一化技术 [15]。

B. Spectral Normalization

谱归一化通过约束目标函数的李普希茨常数来稳定深度神经网络的训练。研究表明，经过谱归一化的深度神经网络也具有好的泛化能力 [18]，这表明了机器学习中的稳定性。从数学上讲，函数 $\|f\|_{\text{Lip}}$ 的李普希茨常数被定义为满足以下条件的最小值

$$\forall \mathbf{x}, \mathbf{x}' : \|f(\mathbf{x}) - f(\mathbf{x}')\|_2 / \|\mathbf{x} - \mathbf{x}'\|_2 \leq \|f\|_{\text{Lip}}.$$

已知，一个一般可微函数的利普希茨常数是其梯度在其定义域上的最大谱范数（最大奇异值） $\|f\|_{\text{Lip}} = \sup_{\mathbf{x}} \sigma(\nabla f(\mathbf{x}))$ 。

(3) 中的 ReLU 深度神经网络是函数的组合。因此，我们可以通过约束每一层的谱范数来界定网络的 Lipschitz 常数 $g^l(\mathbf{x}) = \phi(W^l \mathbf{x})$ 。因此，对于线性映射 $g(\mathbf{x}) = W\mathbf{x}$ ，每一层的谱范数由 $\|g\|_{\text{Lip}} = \sup_{\mathbf{x}} \sigma(\nabla g(\mathbf{x})) = \sup_{\mathbf{x}} \sigma(W) = \sigma(W)$ 给出。利用 ReLU 激活函数的 Lipschitz 范数 $\phi(\cdot)$ 等于 1 的事实，并结合不等式 $\|g_1 \circ g_2\|_{\text{Lip}} \leq \|g_1\|_{\text{Lip}} \cdot \|g_2\|_{\text{Lip}}$ ，我们可以得到 $\|f\|_{\text{Lip}}$ 的下界：

$$\|f\|_{\text{Lip}} \leq \|g^{L+1}\|_{\text{Lip}} \cdot \|\phi\|_{\text{Lip}} \cdots \|g^1\|_{\text{Lip}} = \prod_{l=1}^{L+1} \sigma(W^l). \quad (4)$$

在实践中，我们可以在训练期间将谱归一化应用于每一层的权重矩阵，如下所示：

$$\bar{W} = W / \sigma(W) \cdot \gamma^{\frac{1}{L+1}}, \quad (5)$$

其中 γ 是 DNN 的预期 Lipschitz 常数。下面的引理给出了具有谱归一化的 ReLU DNN 的 Lipschitz 常数的界。

Lemma 3.1: 对于一个多层 ReLU 网络 $f(\mathbf{x}, \boldsymbol{\theta})$ ，如公式 (3) 定义的那样，输出层没有激活函数。使用谱归一化，整个网络的 Lipschitz 常数满足：

$$\|f(\mathbf{x}, \bar{\boldsymbol{\theta}})\|_{\text{Lip}} \leq \gamma,$$

使用光谱归一化的参数 $\bar{\boldsymbol{\theta}} = \bar{W}^1, \dots, \bar{W}^{L+1}$ 。

Proof: 如同在 (4) 中，Lipschitz 常数可以写成所有层的谱范数的组合。证明遵循在 (5) 中约束的谱范数。 ■

¹Demo videos: <https://youtu.be/FLLSG0S78ik>

C. Constrained Training

We apply gradient-based optimization to train the ReLU DNN with a bounded Lipschitz constant. Estimating $\mathbf{f}_a$ in (1) boils down to optimizing the parameters $\boldsymbol{\theta}$ in the ReLU network in (3), given the observed value of $\mathbf{x}$ and the target output. In particular, we want to control the Lipschitz constant of the ReLU network.

The optimization objective is as follows, where we minimize the prediction error with constrained Lipschitz constant:

$$\begin{aligned} & \underset{\boldsymbol{\theta}}{\text{minimize}} && \sum_{t=1}^T \frac{1}{T} \|\mathbf{y}_t - f(\mathbf{x}_t, \boldsymbol{\theta})\|_2 \\ & \text{subject to} && \|f\|_{\text{Lip}} \leq \gamma. \end{aligned} \quad (6)$$

In our case, $\mathbf{y}_t$ is the observed disturbance forces and $\mathbf{x}_t$ is the observed states and control inputs. According to the upper bound in (4), we can substitute the constraint by minimizing the spectral norm of the weights in each layer. We use stochastic gradient descent (SGD) to optimize (6) and apply spectral normalization to regulate the weights. From Lemma 3.1, the trained ReLU DNN has a Lipschitz constant.

IV. NEURAL LANDER CONTROLLER DESIGN

Our *Neural-Lander* controller for 3-D trajectory tracking is constructed as a nonlinear feedback linearization controller whose stability guarantees are obtained using the spectral normalization of the DNN-based ground-effect model. We then exploit the Lipschitz property of the DNN to solve for the resulting control input using fixed-point iteration.

A. Reference Trajectory Tracking

The position tracking error is defined as $\tilde{\mathbf{p}} = \mathbf{p} - \mathbf{p}_d$. Our controller uses a composite variable $\mathbf{s} = 0$ as a manifold on which $\tilde{\mathbf{p}}(t) \rightarrow 0$ exponentially:

$$\mathbf{s} = \dot{\tilde{\mathbf{p}}} + \Lambda \tilde{\mathbf{p}} = \dot{\mathbf{p}} - \mathbf{v}_r \quad (7)$$

with Λ as a positive definite or diagonal matrix. Now the trajectory tracking problem is transformed to tracking a reference velocity $\mathbf{v}_r = \dot{\mathbf{p}}_d - \Lambda \tilde{\mathbf{p}}$.

Using the methods described in Sec. III, we define $\hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})$ as the DNN approximation to the disturbance aerodynamic forces, with $\boldsymbol{\zeta}$ being the partial states used as input features to the network. We design the total desired rotor force $\mathbf{f}_d$ as

$$\mathbf{f}_d = (R\mathbf{f}_u)_d = \bar{\mathbf{f}}_d - \hat{\mathbf{f}}_a, \text{ with } \bar{\mathbf{f}}_d = m\dot{\mathbf{v}}_r - K_v\mathbf{s} - m\mathbf{g}. \quad (8)$$

Substituting (8) into (1), the closed-loop dynamics would simply become $m\dot{\mathbf{s}} + K_v\mathbf{s} = \boldsymbol{\epsilon}$, with approximation error $\boldsymbol{\epsilon} = \mathbf{f}_a - \hat{\mathbf{f}}_a$. Hence, $\tilde{\mathbf{p}}(t) \rightarrow \mathbf{0}$ globally and exponentially with bounded error, as long as $\|\boldsymbol{\epsilon}\|$ is bounded [19]–[21].

Consequently, desired total thrust T_d and desired force direction $\hat{\mathbf{k}}_d$ can be computed as

$$T_d = \mathbf{f}_d \cdot \hat{\mathbf{k}}, \text{ and } \hat{\mathbf{k}}_d = \mathbf{f}_d / \|\mathbf{f}_d\|, \quad (9)$$

with $\hat{\mathbf{k}}$ being the unit vector of rotor thrust direction (typically z -axis in quadrotors). Using $\hat{\mathbf{k}}_d$ and fixing a desired yaw angle, desired attitude R_d can be deduced [22]. We assume that a

nonlinear attitude controller uses the desired torque $\boldsymbol{\tau}_d$ from rotors to track $R_d(t)$. One such example is in [21]:

$$\boldsymbol{\tau}_d = J\dot{\boldsymbol{\omega}}_r - J\boldsymbol{\omega} \times \boldsymbol{\omega}_r - K_\omega(\boldsymbol{\omega} - \boldsymbol{\omega}_r), \quad (10)$$

where the reference angular rate $\boldsymbol{\omega}_r$ is designed similar to (7), so that when $\boldsymbol{\omega} \rightarrow \boldsymbol{\omega}_r$, exponential trajectory tracking of a desired attitude $R_d(t)$ is guaranteed within some bounded error in the presence of bounded disturbance torques.

B. Learning-based Discrete-time Nonlinear Controller

From (2), (9) and (10), we can relate the desired wrench $\boldsymbol{\eta}_d = [T_d, \boldsymbol{\tau}_d^\top]^\top$ with the control signal $\mathbf{u}$ through

$$B_0\mathbf{u} = \boldsymbol{\eta}_d = \begin{bmatrix} (\bar{\mathbf{f}}_d - \hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})) \cdot \hat{\mathbf{k}} \\ \boldsymbol{\tau}_d \end{bmatrix}. \quad (11)$$

Because of the dependency of $\hat{\mathbf{f}}_a$ on $\mathbf{u}$, the control synthesis problem here is non-affine. Therefore, we propose the following fixed-point iteration method for solving (11):

$$\mathbf{u}_k = B_0^{-1}\boldsymbol{\eta}_d(\mathbf{u}_{k-1}), \quad (12)$$

where $\mathbf{u}_k$ and $\mathbf{u}_{k-1}$ are the control input for current and previous time-step in the discrete-time controller. Next, we prove the stability of the system and convergence of the control inputs in (12).

V. NONLINEAR STABILITY ANALYSIS

The closed-loop tracking error analysis provides a direct correlation on how to tune the neural network and controller parameter to improve control performance and robustness.

A. Control Allocation as Contraction Mapping

We first show that the control input $\mathbf{u}_k$ converges to the solution of (11) when all states are fixed.

Lemma 5.1: Define mapping $\mathbf{u}_k = \mathcal{F}(\mathbf{u}_{k-1})$ based on (12) and fix all current states:

$$\mathcal{F}(\mathbf{u}) = B_0^{-1} \begin{bmatrix} (\bar{\mathbf{f}}_d - \hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})) \cdot \hat{\mathbf{k}} \\ \boldsymbol{\tau}_d \end{bmatrix}. \quad (13)$$

If $\hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})$ is L_a -Lipschitz continuous, and $\sigma(B_0^{-1}) \cdot L_a < 1$; then $\mathcal{F}(\cdot)$ is a contraction mapping, and $\mathbf{u}_k$ converges to unique solution of $\mathbf{u}^* = \mathcal{F}(\mathbf{u}^*)$.

Proof: $\forall \mathbf{u}_1, \mathbf{u}_2 \in \mathcal{U}$ with $\mathcal{U}$ being a compact set of feasible control inputs; and given fixed states as $\bar{\mathbf{f}}_d$, $\boldsymbol{\tau}_d$ and $\hat{\mathbf{k}}$, then:

$$\begin{aligned} \|\mathcal{F}(\mathbf{u}_1) - \mathcal{F}(\mathbf{u}_2)\|_2 &= \left\| B_0^{-1} \begin{pmatrix} \hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u}_1) - \hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u}_2) \\ 0 \end{pmatrix} \right\|_2 \\ &\leq \sigma(B_0^{-1}) \cdot L_a \|\mathbf{u}_1 - \mathbf{u}_2\|_2. \end{aligned}$$

Thus, $\exists \alpha < 1$, s.t. $\|\mathcal{F}(\mathbf{u}_1) - \mathcal{F}(\mathbf{u}_2)\|_2 < \alpha \|\mathbf{u}_1 - \mathbf{u}_2\|_2$. Hence, $\mathcal{F}(\cdot)$ is a contraction mapping. $\blacksquare$

C. Constrained Training

我们应用基于梯度的优化来训练具有有界Lipschitz常数的ReLU深度神经网络。估计(1)中的 $\mathbf{f}_a$ 归结为在给定 $\mathbf{x}$ 的观测值和目标输出的情况下优化ReLU网络(3)中的参数 θ 。特别地，我们希望控制ReLU网络的Lipschitz常数。

优化目标如下，我们在受限的利普希茨常数下最小化预测误差：

$$\begin{aligned} & \underset{\theta}{\text{minimize}} && \sum_{t=1}^T \frac{1}{T} \|\mathbf{y}_t - f(\mathbf{x}_t, \theta)\|_2 \\ & \text{subject to} && \|f\|_{\text{Lip}} \leq \gamma. \end{aligned} \quad (6)$$

在我们的例子中， $\mathbf{y}_t$ 是观测到的扰动力， $\mathbf{x}_t$ 是观测到的状态和控制输入。根据公式 (4) 中的上界，我们可以通过最小化每一层权重的谱范数来替代该约束。我们使用随机梯度下降 (SGD) 来优化公式 (6)，并应用谱归一化来调节权重。根据引理 3.1，训练后的 ReLU DNN 具有一个 Lipschitz 常数。

四、神经着陆器控制器设计

我们的 *Neural-Lander* 控制器用于三维轨迹跟踪，被构建为非线性反馈线性化控制器，其稳定性保证是通过基于 DNN 的地面效应模型的谱归一化获得的。然后，我们利用 DNN 的 Lipschitz 特性，通过固定点迭代求解由此产生的控制输入。

A. Reference Trajectory Tracking

位置跟踪误差定义为 $\tilde{\mathbf{p}} = \mathbf{p} - \mathbf{p}_d$ 。我们的控制器使用复合变量 $\mathbf{s} = 0$ 作为一个流形，在其上 $\tilde{\mathbf{p}}(t) \rightarrow 0$ 指数地：

$$\mathbf{s} = \dot{\tilde{\mathbf{p}}} + \Lambda \tilde{\mathbf{p}} = \dot{\mathbf{p}} - \mathbf{v}_r \quad (7)$$

将 Λ 作为正定或对角矩阵。现在，轨迹跟踪问题被转化为跟踪参考速度 $\mathbf{v}_r = \dot{\mathbf{p}}_d - \Lambda \tilde{\mathbf{p}}$ 。

使用第 III 节中描述的方法，我们将 $\hat{\mathbf{f}}_a(\zeta, \mathbf{u})$ 定义为扰动气动力的深度神经网络近似，其中 ζ 是用作网络输入特征的部分状态。我们将总期望转子力 $\mathbf{f}_d$ 设计为

$$\mathbf{f}_d = (R\mathbf{f}_u)_d = \bar{\mathbf{f}}_d - \hat{\mathbf{f}}_a, \text{ with } \bar{\mathbf{f}}_d = m\dot{\mathbf{v}}_r - K_v\mathbf{s} - m\mathbf{g}. \quad (8)$$

将 (8) 代入 (1)，闭环动力学将简化为 $m\dot{\mathbf{s}} + K_v\mathbf{s} = \epsilon$ ，近似误差为 $\epsilon = \mathbf{f}_a - \hat{\mathbf{f}}_a$ 。因此， $\tilde{\mathbf{p}}(t) \rightarrow 0$ 全局且指数地收敛，同时误差有界，只要 $\|\epsilon\|$ 有界 [19]–[21]。

因此，期望的总推力 T_d 和期望的力方向 $\hat{k}_d$ 可以计算为

$$T_d = \mathbf{f}_d \cdot \hat{k}, \text{ and } \hat{k}_d = \mathbf{f}_d / \|\mathbf{f}_d\|, \quad (9)$$

其中 $\hat{k}$ 是转子推力方向的单位向量（在四旋翼中通常是 z -轴）。使用 $\hat{k}_d$ 并固定一个期望的偏航角，可以推导出期望姿态 R_d [22]。我们假设一个

非线性姿态控制器使用来自旋翼的期望转矩 τ_d 来跟踪 $R_d(t)$ 。这样的一个例子见于 [21]：

$$\tau_d = J\dot{\omega}_r - J\omega \times \omega_r - K_\omega(\omega - \omega_r), \quad (10)$$

参考角速度 ω_r 的设计类似于 (7)，因此当 $\omega \rightarrow \omega_r$ 时，在存在有界扰动力矩的情况下，可保证期望姿态 $R_d(t)$ 的指数轨迹跟踪在某个有界误差范围内。

B. Learning-based Discrete-time Nonlinear Controller

从(2)、(9)和(10)中，我们可以通过控制信号 $\mathbf{u}$ 将期望的扳手 $\eta_d = [T_d, \tau_d^\top]^\top$ 关联起来

$$B_0\mathbf{u} = \eta_d = \begin{bmatrix} (\bar{\mathbf{f}}_d - \hat{\mathbf{f}}_a(\zeta, \mathbf{u})) \cdot \hat{k} \\ \tau_d \end{bmatrix}. \quad (11)$$

由于 $\hat{\mathbf{f}}_a$ 对 $\mathbf{u}$ 的依赖，这里的控制综合问题是非仿射的。因此，我们提出以下用于求解 (11) 的不动点迭代方法：

$$\mathbf{u}_k = B_0^{-1}\eta_d(\mathbf{u}_{k-1}), \quad (12)$$

其中 $\mathbf{u}_k$ 和 $\mathbf{u}_{k-1}$ 是离散时间控制器中当前时刻和上一个时刻的控制输入。接下来，我们证明系统的稳定性以及 (12) 中控制输入的收敛性。

五、非线性稳定性分析

闭环跟踪误差分析提供了一个直接的关联，说明如何调整神经网络和控制器参数以提高控制性能和鲁棒性。

A. Control Allocation as Contraction Mapping

我们首先表明，当所有状态固定时，控制输入 $\mathbf{u}_k$ 收敛到 (11) 的解。

Lemma 5.1: 根据 (定义映射 $\mathbf{u}_k = \mathcal{F}(\mathbf{u}_{k-1})$ (12) 并修复所有当前状态：

$$\mathcal{F}(\mathbf{u}) = B_0^{-1} \begin{bmatrix} (\bar{\mathbf{f}}_d - \hat{\mathbf{f}}_a(\zeta, \mathbf{u})) \cdot \hat{k} \\ \tau_d \end{bmatrix}. \quad (13)$$

如果 $\hat{\mathbf{f}}_a(\zeta, \mathbf{u})$ 是 L_a -Lipschitz 连续的，并且 $\sigma(B_0^{-1}) \cdot L_a < 1$ ；那么 $\mathcal{F}(\cdot)$ 是一个收缩映射，并且 $\mathbf{u}_k$ 收敛到 $\mathbf{u}^* = \mathcal{F}(\mathbf{u}^*)$ 的唯一解。

Proof: $\forall \mathbf{u}_1, \mathbf{u}_2 \in \mathcal{U}$ ，其中 $\mathcal{U}$ 是一个可行控制输入的紧集；并且给定固定状态为 $\bar{\mathbf{f}}_d$ 、 τ_d 和 $\hat{k}$ ，那么：

$$\begin{aligned} \|\mathcal{F}(\mathbf{u}_1) - \mathcal{F}(\mathbf{u}_2)\|_2 &= \|B_0^{-1} (\hat{\mathbf{f}}_a(\zeta, \mathbf{u}_1) - \hat{\mathbf{f}}_a(\zeta, \mathbf{u}_2))\|_2 \\ &\leq \sigma(B_0^{-1}) \cdot L_a \|\mathbf{u}_1 - \mathbf{u}_2\|_2. \end{aligned}$$

因此， $\exists \alpha < 1$ ，使得 $\|\mathcal{F}(\mathbf{u}_1) - \mathcal{F}(\mathbf{u}_2)\|_2 < \alpha \|\mathbf{u}_1 - \mathbf{u}_2\|_2$ 。因此， $\mathcal{F}(\cdot)$ 是一个压缩映射。

B. Stability of Learning-based Nonlinear Controller

Before continuing to prove the stability of the full system, we make the following assumptions.

Assumption 1: The desired states along the position trajectory $\mathbf{p}_d(t)$, $\dot{\mathbf{p}}_d(t)$, and $\ddot{\mathbf{p}}_d(t)$ are bounded.

Assumption 2: One-step difference of control signal satisfies $\|\mathbf{u}_k - \mathbf{u}_{k-1}\| \leq \rho \|\mathbf{s}\|$ with a small positive ρ .

Here we provide the intuition behind this assumption. From (13), we can derive the following approximate relation with $\Delta(\cdot)_k = \|(\cdot)_k - (\cdot)_{k-1}\|$:

$$\Delta u_k \leq \sigma(B_0^{-1})(L_a \Delta u_{k-1} + L_a \Delta \zeta_k + m \Delta \dot{v}_{r,k} + \lambda_{\max}(K_v) \Delta s_k + \Delta \tau_{d,k}).$$

Because update rate of attitude controller (> 100 Hz) and motor speed control (> 5 kHz) are much higher than that of the position controller (≈ 10 Hz), in practice, we can safely neglect Δs_k , $\Delta \dot{v}_{r,k}$, and $\Delta \zeta_k$ in one update (Theorem 11.1 [23]). Furthermore, $\Delta \tau_{d,k}$ can be limited internally by the attitude controller. It leads to:

$$\Delta u_k \leq \sigma(B_0^{-1})(L_a \Delta u_{k-1} + c),$$

with c being a small constant and $\sigma(B_0^{-1}) \cdot L_a < 1$ from Lemma. 5.1, we can deduce that Δu rapidly converges to a small ultimate bound between each position controller update.

Assumption 3: The learning error of $\hat{\mathbf{f}}_a(\zeta, \mathbf{u})$ over the compact sets $\zeta \in \mathcal{Z}$, $\mathbf{u} \in \mathcal{U}$ is upper bounded by $\epsilon_m = \sup_{\zeta \in \mathcal{Z}, \mathbf{u} \in \mathcal{U}} \|\epsilon(\zeta, \mathbf{u})\|$, where $\epsilon(\zeta, \mathbf{u}) = \mathbf{f}_a(\zeta, \mathbf{u}) - \hat{\mathbf{f}}_a(\zeta, \mathbf{u})$.

DNNs have been shown to generalize well to the set of unseen events that are from almost the same distribution as training set [24], [25]. This empirical observation is also theoretically studied in order to shed more light toward an understanding of the complexity of these models [18], [26]–[28]. Based on the above assumptions, we can now present our overall stability and robustness result.

Theorem 5.2: Under Assumptions 1-3, for a time-varying $\mathbf{p}_d(t)$, the controller defined in (8) and (12) with $\lambda_{\min}(K_v) > L_a \rho$ achieves exponential convergence of composite variable $\mathbf{s}$ to error ball $\lim_{t \rightarrow \infty} \|\mathbf{s}(t)\| = \epsilon_m / (\lambda_{\min}(K_v) - L_a \rho)$ with rate $((\lambda_{\min}(K_v) - L_a \rho) / m)$. And $\tilde{\mathbf{p}}$ exponentially converges to error ball

$$\lim_{t \rightarrow \infty} \|\tilde{\mathbf{p}}(t)\| = \frac{\epsilon_m}{\lambda_{\min}(\Lambda)(\lambda_{\min}(K_v) - L_a \rho)} \quad (14)$$

with rate $\lambda_{\min}(\Lambda)$.

Proof: We begin the proof by selecting a Lyapunov function as $\mathcal{V}(\mathbf{s}) = \frac{1}{2} m \|\mathbf{s}\|^2$, then by applying the controller (8), we get the time-derivative of $\mathcal{V}$:

$$\begin{aligned} \dot{\mathcal{V}} &= \mathbf{s}^\top (-K_v \mathbf{s} + \hat{\mathbf{f}}_a(\zeta_k, \mathbf{u}_k) - \hat{\mathbf{f}}_a(\zeta_k, \mathbf{u}_{k-1}) + \epsilon(\zeta_k, \mathbf{u}_k)) \\ &\leq -\mathbf{s}^\top K_v \mathbf{s} + \|\mathbf{s}\| (\|\hat{\mathbf{f}}_a(\zeta_k, \mathbf{u}_k) - \hat{\mathbf{f}}_a(\zeta_k, \mathbf{u}_{k-1})\| + \epsilon_m) \end{aligned}$$

Let $\lambda = \lambda_{\min}(K_v)$ denote the minimum eigenvalue of the positive-definite matrix K_v . By applying the Lipschitz property of $\hat{\mathbf{f}}_a$ theorem 3.1 and Assumption 2, we obtain

$$\dot{\mathcal{V}} \leq -\frac{2(\lambda - L_a \rho)}{m} \mathcal{V} + \sqrt{\frac{2\mathcal{V}}{m}} \epsilon_m$$

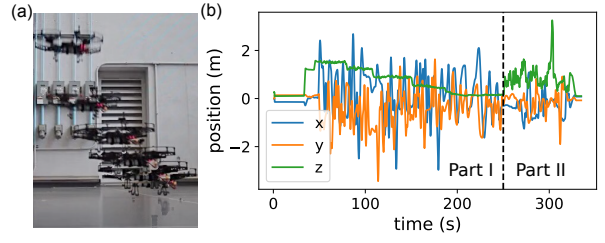


Fig. 1: (a) Intel Aero drone; (b) Training data trajectory. Part I (0 to 250s) contains maneuvers at different heights (0.05 m to 1.50 m). Part II (250 s to 350 s) includes random x , y , and z motions for maximum state-space coverage.

Using the Comparison Lemma [23], we define $\mathcal{W}(t) = \sqrt{\mathcal{V}(t)} = \sqrt{m/2} \|\mathbf{s}\|$ and $\dot{\mathcal{W}} = \dot{\mathcal{V}} / (2\sqrt{\mathcal{V}})$ to obtain

$$\|\mathbf{s}(t)\| \leq \|\mathbf{s}(t_0)\| \exp\left(-\frac{\lambda - L_a \rho}{m}(t - t_0)\right) + \frac{\epsilon_m}{\lambda - L_a \rho}$$

It can be shown that this leads to finite-gain $\mathcal{L}_p$ stability and input-to-state stability (ISS) [29]. Furthermore, the hierarchical combination between $\mathbf{s}$ and $\tilde{\mathbf{p}}$ in (7) results in $\lim_{t \rightarrow \infty} \|\tilde{\mathbf{p}}(t)\| = \lim_{t \rightarrow \infty} \|\mathbf{s}(t)\| / \lambda_{\min}(\Lambda)$, yielding (14). ■

VI. EXPERIMENTS

In our experiments, we evaluate both the generalization performance of our DNN as well as the overall control performance of *Neural-Lander*. The experimental setup is composed of a motion capture system with 17 cameras, a WiFi router for communication, and an Intel Aero drone, weighing 1.47 kg with an onboard Linux computer (2.56 GHz Intel Atom x7 processor, 4 GB DDR3 RAM). We retrofitted the drone with eight reflective infrared markers for accurate position, attitude and velocity estimation at 100Hz. The Intel Aero drone and the test space are shown in Fig. 1(a).

A. Bench Test

To identify a good nominal model, we first measured the mass, m , diameter of the rotor, D , the air density, ρ , gravity, g . Then we performed bench test to determine the thrust constant, c_T , as well as the non-dimensional thrust coefficient $C_T = \frac{c_T}{\rho D^4}$. Note that C_T is a function of propeller speed n , and here we picked a nominal value at $n = 2000$ RPM.

B. Real-World Flying Data and Preprocessing

To estimate the disturbance force $\mathbf{f}_a$, an expert pilot manually flew the drone at different heights, and we collected training data consisting of sequences of state estimates and control inputs $\{(\mathbf{p}, \mathbf{v}, R, \mathbf{u}), \mathbf{y}\}$ where $\mathbf{y}$ is the observed value of $\mathbf{f}_a$. We utilized the relation $\mathbf{f}_a = m\dot{\mathbf{v}} - m\mathbf{g} - R\mathbf{f}_u$ from (1) to calculate $\mathbf{f}_a$, where $\mathbf{f}_u$ is calculated based on the nominal c_T from the bench test in Sec. VI-A. Our training set is a single continuous trajectory with varying heights and velocities. The trajectory has two parts shown in Fig. 1(b). We aim to learn the ground effect through Part I of the training set, and other aerodynamics forces such as air drag through Part II.

B. Stability of Learning-based Nonlinear Controller

在继续证明整个系统的稳定性之前，我们做出以下假设。

Assumption 1: 沿位置轨迹 $\mathbf{p}_d(t)$ 、 $\dot{\mathbf{p}}_d(t)$ 和 $\ddot{\mathbf{p}}_d(t)$ 的期望状态是有界的。

Assumption 2: 控制信号的一步差分满足 $\|\mathbf{u}_k - \mathbf{u}_{k-1}\| \leq \rho \|\mathbf{s}\|$ ，具有一个小的正 ρ 。

这里我们提供这一假设背后的直觉。从 (13) 式中，我们可以得出与 $\Delta(\cdot)_k = \|(\cdot)_k - (\cdot)_{k-1}\|$ 的以下近似关系：

$$\Delta u_k \leq \sigma(B_0^{-1})(L_a \Delta u_{k-1} + L_a \Delta \zeta_k + m \Delta \dot{v}_{r,k} + \lambda_{\max}(K_v) \Delta s_k + \Delta \tau_{d,k}).$$

由于姿态控制器的更新速率 (> 100 Hz) 和电机速度控制 (> 5 kHz) 远高于位置控制器 (≈ 10 Hz) 的更新速率，在实践中，我们可以在一次更新中安全地忽略 Δs_k 、 $\Delta \dot{v}_{r,k}$ 和 $\Delta \zeta_k$ (定理 11.1 [23])。此外， $\Delta \tau_{d,k}$ 可以由姿态控制器在内部加以限制。这导致：

$$\Delta u_k \leq \sigma(B_0^{-1})(L_a \Delta u_{k-1} + c),$$

由于 c 是一个小常数且 $\sigma(B_0^{-1}) \cdot L_a < 1$ 是引理 5.1 中的 1，我们可以推导出 Δu 在每次位置控制器更新之间迅速收敛到一个小的最终界限。

Assumption 3: $\hat{\mathbf{f}}_a(\zeta, \mathbf{u})$ 在紧集 $\zeta \in \mathcal{Z}, \mathbf{u} \in \mathcal{U}$ 上的学习误差被 $\epsilon_m = \sup_{\zeta \in \mathcal{Z}, \mathbf{u} \in \mathcal{U}} \|\epsilon(\zeta, \mathbf{u})\|$ 上界，其中 $\epsilon(\zeta, \mathbf{u}) = \mathbf{f}_a(\zeta, \mathbf{u}) - \hat{\mathbf{f}}_a(\zeta, \mathbf{u})$ 。

已有研究表明，深度神经网络 (DNN) 能够很好地推广到来自几乎与训练集相同分布的未见事件集合 [24], [25]。这一经验观察也在理论上得到了研究，以便更清楚地理解这些模型的复杂性 [18], [26]–[28]。基于上述假设，我们现在可以呈现我们的整体稳定性和鲁棒性结果。

Theorem 5.2: 在假设 1-3 下，对于时间变化的 $\mathbf{p}_d(t)$ ，由 (8) 和 (12) 定义的控制器和 $\lambda_{\min}(K_v) > L_a \rho$ 实现组合变量 $\mathbf{s}$ 以速率 $((\lambda_{\min}(K_v) - L_a \rho) / m)$ 指数收敛到误差球 $\lim_{t \rightarrow \infty} \|\mathbf{s}(t)\| = \epsilon_m / (\lambda_{\min}(K_v) - L_a \rho)$ 。并且 $\tilde{\mathbf{p}}$ 指数收敛到误差球

$$\lim_{t \rightarrow \infty} \|\tilde{\mathbf{p}}(t)\| = \frac{\epsilon_m}{\lambda_{\min}(\Lambda)(\lambda_{\min}(K_v) - L_a \rho)} \quad (14)$$

以速率 $\lambda_{\min}(\Lambda)$ 。

Proof: 我们通过选择李雅普诺夫函数 $\mathcal{V}(\mathbf{s}) = \frac{1}{2} m \|\mathbf{s}\|^2$ 开始证明，然后通过应用控制器 (8)，我们得到 $\dot{\mathcal{V}}$ 的时间导数：

$$\begin{aligned} \dot{\mathcal{V}} &= \mathbf{s}^\top (-K_v \mathbf{s} + \hat{\mathbf{f}}_a(\zeta_k, \mathbf{u}_k) - \hat{\mathbf{f}}_a(\zeta_k, \mathbf{u}_{k-1}) + \epsilon(\zeta_k, \mathbf{u}_k)) \\ &\leq -\mathbf{s}^\top K_v \mathbf{s} + \|\mathbf{s}\| (\|\hat{\mathbf{f}}_a(\zeta_k, \mathbf{u}_k) - \hat{\mathbf{f}}_a(\zeta_k, \mathbf{u}_{k-1})\| + \epsilon_m) \end{aligned}$$

设 $\lambda = \lambda_{\min}(K_v)$ 表示正定矩阵 K_v 的最小特征值。通过应用 $\hat{\mathbf{f}}_a$ 定理 3.1 的 Lipschitz 性质和假设 2，我们得到

$$\dot{\mathcal{V}} \leq -\frac{2(\lambda - L_a \rho)}{m} \mathcal{V} + \sqrt{\frac{2\mathcal{V}}{m}} \epsilon_m$$

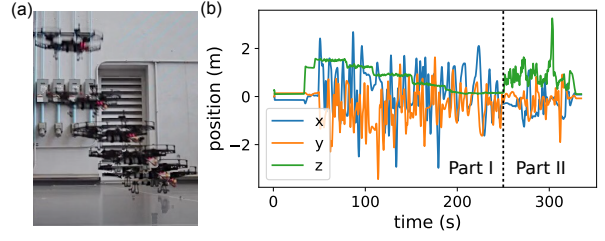


图 1: (a) Intel Aero 无人机; (b) 训练数据轨迹。第 I 部分 (0 到 250 秒) 包含不同高度 (0.05 米到 1.50 米) 的机动动作。第 II 部分 (250 秒到 350 秒) 包括随机的 x 、 y 和 z 动作，以实现最大状态空间覆盖。

Using the Comparison Lemma [23], we define $\mathcal{W}(t) = \sqrt{\mathcal{V}(t)} = \sqrt{m/2} \|\mathbf{s}\|$ and $\dot{\mathcal{W}} = \dot{\mathcal{V}} / (2\sqrt{\mathcal{V}})$ to obtain

$$\|\mathbf{s}(t)\| \leq \|\mathbf{s}(t_0)\| \exp\left(-\frac{\lambda - L_a \rho}{m}(t - t_0)\right) + \frac{\epsilon_m}{\lambda - L_a \rho}$$

可以证明，这导致有限增益 $\mathcal{L}_p$ 稳定性和输入到状态稳定性 (ISS) [29]。此外，(7) 中 $\mathbf{s}$ 和 $\tilde{\mathbf{p}}$ 之间的分层组合导致 $\lim_{t \rightarrow \infty} \|\tilde{\mathbf{p}}(t)\| = \lim_{t \rightarrow \infty} \|\mathbf{s}(t)\| / \lambda_{\min}(\Lambda)$ ，从而得到 (14)。

VI. 实验

在我们的实验中，我们评估了 DNN 的泛化性能以及 *Neural-Lander* 的整体控制性能。实验装置由一个包含 17 台相机的动作捕捉系统、用于通信的 WiFi 路由器以及一架 Intel Aero 无人机组成，该无人机重 1.47 千克，配备有车载 Linux 计算机 (2.56 GHz Intel Atom x7 处理器，4 GB DDR3 内存)。我们在无人机上改装了八个反光红外标记，以实现 100 Hz 的精确位置、姿态和速度估计。Intel Aero 无人机和测试空间如图 1(a) 所示。

A. Bench Test

为了确定一个好的标称模型，我们首先测量了质量 m 、转子的直径 D 、空气密度 ρ 、重力 g 。然后我们进行了台架测试以确定推力常数 c_T ，以及无量纲推力系数 $C_T = \frac{c_T}{\rho D^4 \omega^3}$ 。注意， C_T 是螺旋桨转速 n 的函数，这里我们选择了在 2000 转/分钟 $n = 100$ 时的标称值。

B. Real-World Flying Data and Preprocessing

为了估计扰动力 $\mathbf{f}_a$ ，一名经验丰富的飞行员在不同高度手动操纵无人机飞行，我们收集了由状态估计序列和控制输入 $\{(\mathbf{p}, \mathbf{v}, R, \mathbf{u}), \mathbf{y}\}$ 组成的训练数据，其中 $\mathbf{y}$ 是 $\mathbf{f}_a$ 的观测值。我们利用公式 (1) 中的关系 $\mathbf{f}_a = m\dot{\mathbf{v}} - m\mathbf{g} - R\mathbf{f}_u$ 计算 $\mathbf{f}_a$ ，其中 $\mathbf{f}_u$ 是基于第 VI-A 节台架测试中的标称 c_T 计算得到的。我们的训练集是一个具有不同高度和速度的连续轨迹。轨迹分为两部分，如图 1(b) 所示。我们旨在通过训练集的第一部分学习地面效应，通过第二部分学习其他空气动力学力，例如空气阻力。

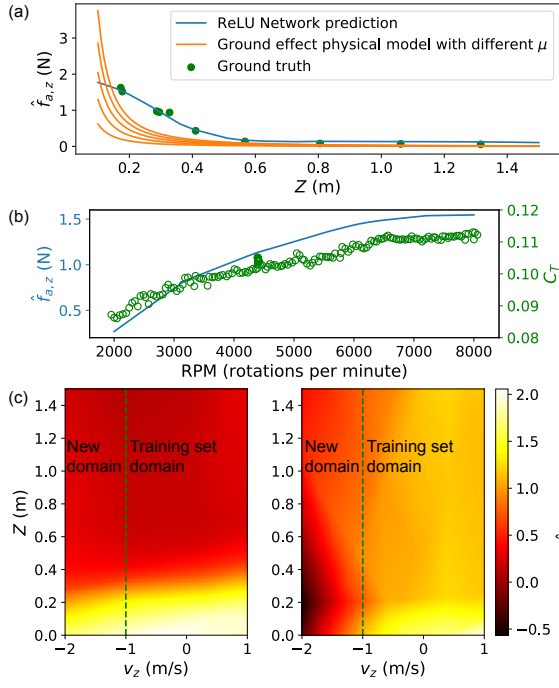


Fig. 2: (a) Learned $\hat{f}_{a,z}$ compared to the ground effect model with respect to height z , with $v_z = v_x = v_y = 0$ m/s, $R = I$, $\mathbf{u} = 6400$ RPM. Ground truth points are from hovering data at different heights. (b) Learned $\hat{f}_{a,z}$ with respect to rotation speed n ($z = 0.2$ m, $v_z = 0$ m/s), compared to C_T measured in the bench test. (c) Heatmaps of learned $\hat{f}_{a,z}$ versus z and v_z . (Left) ReLU network with spectral normalization. (Right) ReLU network without spectral normalization.

C. DNN Prediction Performance

We train a deep ReLU network $\hat{\mathbf{f}}_a(\zeta, \mathbf{u}) = \hat{\mathbf{f}}_a(z, \mathbf{v}, R, \mathbf{u})$, with $z, \mathbf{v}, R, \mathbf{u}$ corresponding to global height, global velocity, attitude, and control input. We build the ReLU network using PyTorch [30]. Our ReLU network consists of four fully-connected hidden layers, with input and the output dimensions 12 and 3, respectively. We use spectral normalization (5) to constrain the Lipschitz constant of the DNN.

We compare the near-ground estimation accuracy our DNN model with existing 1D steady ground effect model [1], [3]:

$$T(n, z) = \frac{n^2}{1 - \mu(\frac{D}{8z})^2} c_T(n) = n^2 c_T(n_0) + \bar{f}_{a,z}, \quad (15)$$

where T is the thrust generated by propellers, n is the rotation speed, n_0 is the idle RPM, and μ depends on the number and the arrangement of propellers ($\mu = 1$ for a single propeller, but must be tuned for multiple propellers). Note that c_T is a function of n . Thus, we can derive $\bar{f}_{a,z}(n, z)$ from $T(n, z)$.

Fig. 2(a) shows the comparison between the estimated $\mathbf{f}_a$ from DNN and the theoretical ground effect model (15) at different z (assuming $T = mg$ when $z = \infty$). We can see that our DNN can achieve much better estimates than the theoretical ground effect model. We further investigate the trend of $\bar{f}_{a,z}$ with respect to the rotation speed n . Fig. 2(b) shows the learned $\hat{f}_{a,z}$ over the rotation speed n at a given

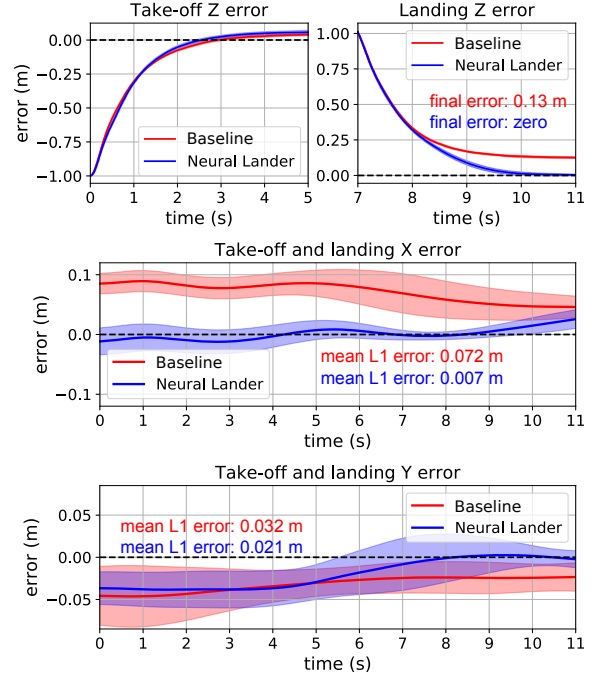


Fig. 3: Baseline Controller and *Neural-Lander* performance in take-off and landing. Means (solid curves) and standard deviations (shaded areas) of 10 trajectories.

height, in comparison with the C_T measured from the bench test. We observe that the increasing trend of the estimates $\hat{f}_{a,z}$ is consistent with bench test results for C_T .

To understand the benefits of SN, we compared $\hat{f}_{a,z}$ predicted by the DNNs trained both with and without SN as shown in Fig. 2(c). Note that v_z from -1 m/s to 1 m/s is covered in our training set, but -2 m/s to -1 m/s is not. We observe the following differences:

- 1) Ground effect: $\hat{f}_{a,z}$ increases as z decreases, which is also shown in Fig. 2(a).
- 2) Air drag: $\hat{f}_{a,z}$ increases as the drone goes down ($v_z < 0$) and it decreases as the drone goes up ($v_z > 0$).
- 3) Generalization: the spectral normalized DNN is much smoother and can also generalize to new input domains not contained in the training set.

In [18], the authors theoretically show that spectral normalization can provide tighter generalization guarantees on unseen data, which is consistent with our empirical observation. We will connect generalization theory more tightly with our robustness guarantees in the future.

D. Baseline Controller

We compared the *Neural-Lander* with a Baseline Nonlinear Tracking Controller. We implemented both a Baseline Controller similar to (7) and (8) with $\hat{\mathbf{f}}_a \equiv 0$, as well as an integral controller variation with $\mathbf{v}_r = \dot{\mathbf{p}}_d - 2\Lambda\tilde{\mathbf{p}} - \Lambda^2 \int_0^t \tilde{\mathbf{p}}(\tau) d\tau$. Though an integral gain can cancel steady-state error during set-point regulation, our flight results showed that the performance can be sensitive to the integral gain,

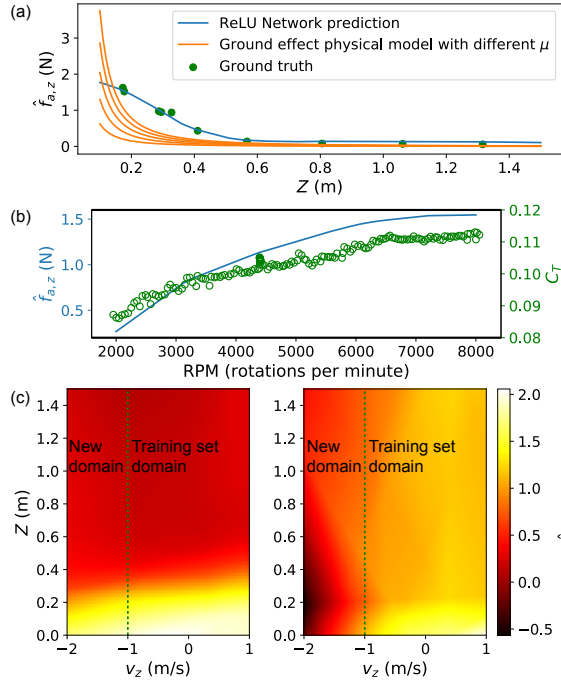


图2: (a) 学习的 $\hat{f}_{a,z}$ 与地面效应模型相比随高度 z 的变化, $v_z = v_x = v_y = 0$ m/s, $R = I$, $\mathbf{u} = 6400$ RPM。真实数据点来自不同高度的悬停数据。(b) 学习的 $\hat{f}_{a,z}$ 随转速 n ($z = 0.2$ m, $v_z = 0$ m/s) 的变化, 与在台架测试中测得的 C_T 相比。(c) 学习的 $\hat{f}_{a,z}$ 相对于 z 和 v_z 的热力图。(左) 带谱归一化的 ReLU 网络。(右) 不带谱归一化的 ReLU 网络。

C. DNN Prediction Performance

我们训练了一个深度 ReLU 网络

$\hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u}) = \hat{\mathbf{f}}_a(z, \mathbf{v}, R, \mathbf{u})$, 其中 z 、 $\mathbf{v}$ 、 R 、 $\mathbf{u}$ 分别对应全局高度、全局速度、姿态和控制输入。我们使用 PyTorch [30] 构建了 ReLU 网络。我们的 ReLU 网络由四个全连接隐藏层组成, 输入和输出维度分别为 12 和 3。我们使用谱归一化 (5) 来约束深度神经网络的 Lipschitz 常数。

我们将我们 DNN 模型的近地面估计精度与现有的一维稳态地面效应模型 [1], [3] 进行比较:

$$T(n, z) = \frac{n^2}{1 - \mu(\frac{D}{8z})^2} c_T(n) = n^2 c_T(n_0) + \bar{f}_{a,z}, \quad (15)$$

其中 T 是螺旋桨产生的推力, n 是旋转速度, n_0 是空转转速, μ 取决于螺旋桨的数量和排列方式 (单个螺旋桨为 $\mu = 1$, 但多螺旋桨需要调节)。注意 c_T 是 n 的函数。因此, 我们可以从 $T(n, z)$ 推导出 $\bar{f}_{a,z}(n, z)$ 。

图2(a) 显示了在不同 z (假设 $T = mg$ 当 $z = \infty$) 时, DNN 估计的 $\mathbf{f}_a$ 与理论地面效应模型(15) 的比较。我们可以看到, 我们的 DNN 可以比理论地面效应模型获得更好的估计。我们进一步研究了 $\bar{f}_{a,z}$ 相对于旋转速度 n 的趋势。图2(b) 显示了在给定条件下旋转速度 n 上学习到的 $\hat{f}_{a,z}$ 。

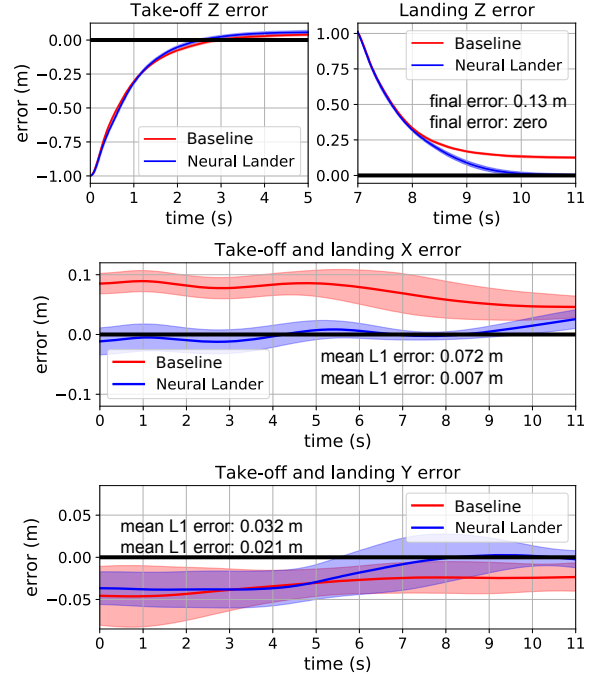


图3: 起飞和着陆时基线控制器和 *Neural-Lander* 的性能。10条轨迹的均值 (实线) 和标准差 (阴影区域)。

高度, 与从台架测试中测得的 C_T 相比。我们观察到, 估计值 $\hat{f}_{a,z}$ 的上升趋势与 C_T 的台架测试结果一致。

为了理解 SN 的益处, 我们比较了图 2(c) 所示由分别采用和未采用 SN 训练的 DNN 预测的 $\hat{f}_{a,z}$ 。请注意, 我们的训练集覆盖了从 -1 m/s 到 1 m/s 的 v_z , 但不包括从 -2 m/s 到 -1 m/s 的 $-$ 。我们观察到以下差异:

- 1) 地效: 当 z 减小时, $\hat{f}_{a,z}$ 增加, 这在图 2(a) 中也有显示。
- 2) 空气阻力: 当无人机下降时 ($v_z < 0$), $\hat{f}_{a,z}$ 增加; 当无人机上升时 ($v_z > 0$), $\hat{f}_{a,z}$ 减少。
- 3) 泛化能力: 谱归一化 DNN 更加平滑, 并且可以泛化到训练集中未包含的新输入域。

在[18]中, 作者从理论上表明频谱归一化可以在未见数据上提供更严格的泛化保证, 这与我们的实证观察一致。我们将来会将泛化理论与我们的鲁棒性保证更紧密地联系起来。

D. Baseline Controller

我们将 *Neural-Lander* 与基线非线性跟踪控制器进行了比较。我们实现了一个与 (7) 和 (8) 类似的基线控制器, $\hat{\mathbf{f}}_a \equiv 0$, 以及一个积分控制器的变体, $\mathbf{v}_r = \dot{\mathbf{p}}_d - 2\Lambda\tilde{\mathbf{p}} - \Lambda^2 \int_0^t \tilde{\mathbf{p}}(\tau) d\tau$ 。虽然积分增益可以在设定调节期间消除稳态误差, 但我们的飞行结果显示, 性能可能对积分增益敏感,

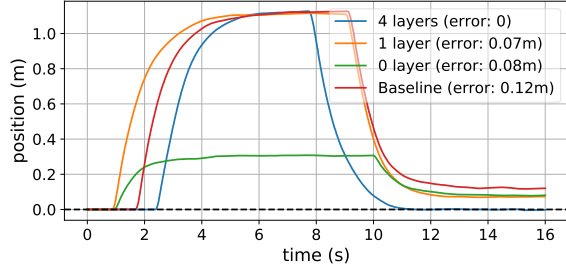


Fig. 4: *Neural-Lander* performance in take-off and landing with different DNN capacities. 1 layer means $\hat{\mathbf{f}}_a = \mathbf{A}\mathbf{x} + \mathbf{b}$; 0 layer means $\hat{\mathbf{f}}_a = \mathbf{b}$; Baseline means $\hat{\mathbf{f}}_a \equiv \mathbf{0}$.

especially during trajectory tracking. This can be seen in the demo video.²

E. Setpoint Regulation Performance

First, we tested the two controllers' performance in take-off/landing, by commanding position setpoint $\mathbf{p}_d$, from $(0, 0, 0)$, to $(0, 0, 1)$, then back to $(0, 0, 0)$, with $\dot{\mathbf{p}}_d \equiv \mathbf{0}$. From Fig. 3, we can conclude that there are two main benefits of our *Neural-Lander*. (a) *Neural-Lander* can control the drone to precisely and smoothly land on the ground surface while the Baseline Controller struggles to achieve 0 terminal height due to the ground effect. (b) *Neural-Lander* can mitigate drifts in $x - y$ plane, as it also learned about additional aerodynamics such as air drag.

Second, we tested *Neural-Lander* performance with different DNN capacities. Fig. 4 shows that compared to the baseline ($\hat{\mathbf{f}}_a \equiv \mathbf{0}$), 1 layer model could decrease z error but it is not enough to land the drone. 0 layer model generated significant error during take-off.

In experiments, we observed the *Neural-Lander* without spectral normalization can even result in unexpected controller outputs leading to crash, which empirically implies the necessity of SN in training the DNN and designing the controller.

F. Trajectory Tracking Performance

To show that our algorithm can handle more complicated environments where physics-based modelling of dynamics would be substantially more difficult, we devise a task of tracking an elliptic trajectory very close to a table with a period of 10 seconds shown in Fig. 5. The trajectory is partially over the table with significant ground effects, and a sharp transition to free space at the edge of the table. We compared the performance of both *Neural-Lander* and Baseline Controller on this test.

In order to model the complex dynamics near the table, we manually flew the drone in the space close to the table to collect another data set. We trained a new ReLU DNN model with $x-y$ positions as additional input features: $\hat{\mathbf{f}}_a(\mathbf{p}, \mathbf{v}, R, \mathbf{u})$. Similar to the setpoint experiment, the benefit of spectral

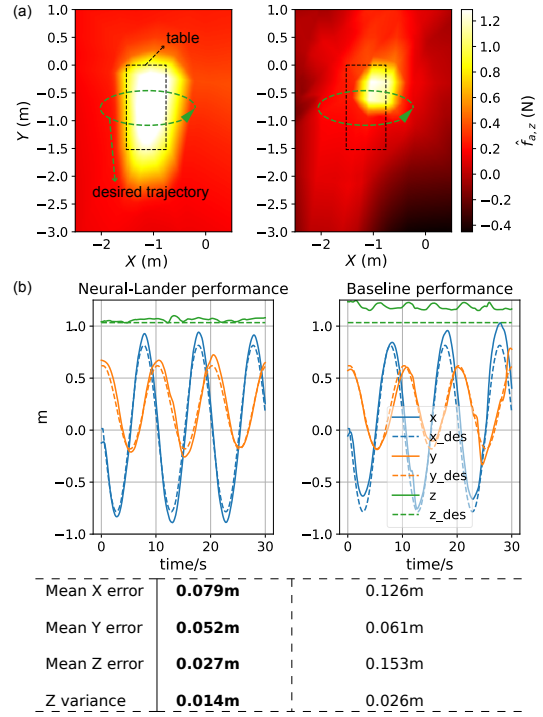


Fig. 5: (a) Heatmaps of learned $\hat{f}_{a,z}$ versus x and y , with other inputs fixed. (Left) ReLU network with spectral normalization. (Right) ReLU network without spectral normalization. (b) Tracking performance and statistics.

normalization can be seen in Fig. 5(a), where only the spectrally-normalized DNN exhibits a clear table boundary.

Fig. 5(b) shows that *Neural-Lander* outperformed the Baseline Controller for tracking the desired position trajectory in all x , y , and z axes. Additionally, *Neural-Lander* showed a lower variance in height, even at the edge of the table, as the controller captured the changes in ground effects when the drone flew over the table.

In summary, the experimental results with multiple ground interaction scenarios show that much smaller tracking errors are obtained by *Neural-Lander*, which is essentially the nonlinear tracking controller with feedforward cancellation of a spectrally-normalized DNN.

VII. CONCLUSIONS

In this paper, we present *Neural-Lander*, a deep learning based nonlinear controller with guaranteed stability for precise quadrotor landing. Compared to the Baseline Controller, *Neural-Lander* is able to significantly improve control performance. The main benefits are (1) our method can learn from coupled unsteady aerodynamics and vehicle dynamics to provide more accurate estimates than theoretical ground effect models, (2) our model can capture both the ground effect and other non-dominant aerodynamics and outperforms the conventional controller in all axes (x , y and z), and (3) we provide rigorous theoretical analysis of our method and guarantee the stability of the controller, which also implies generalization to unseen domains.

²Demo videos: <https://youtu.be/FLlsG0S781k>

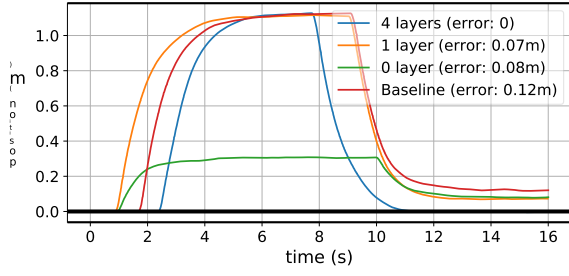


图4: 不同DNN容量下 $Neural-Lander$ 在起飞和着陆中的性能。1层表示 $\hat{\mathbf{f}}_a = \mathbf{A}\mathbf{x} + \mathbf{b}$; 0层表示 $\hat{\mathbf{f}}_a = \mathbf{b}$; 基线表示 $\hat{\mathbf{f}}_a \equiv 0$ 。

尤其是在轨迹跟踪过程中。这可以在演示视频中看到。²

E. Setpoint Regulation Performance

首先, 我们通过命令位置设定点 $\mathbf{p}_d$ 来测试两个控制器在起飞/降落中的性能, 从 $(0, 0, 0)$ 到 $(0, 0, 1)$, 然后再回到 $(0, 0, 0)$, $\mathbf{p}_d \equiv 0$ 。从图 3 中, 我们可以得出我们的 $Neural-Lander$ 有两个主要优点。(a) $Neural-Lander$ 能够控制无人机精确且平稳地着陆在地面上, 而基线控制器由于地面效应难以实现零终端高度。(b) $Neural-Lander$ 可以减轻 $x-y$ 平面上的漂移, 因为它还学习了额外的空气动力学特性, 如空气阻力。

其次, 我们测试了 $Neural-Lander$ 在不同 DNN 容量下的性能。图 4 显示, 相比基线 ($\hat{\mathbf{f}}_a \equiv 0$), 1 层模型可以减少 z 错误, 但这不足以让无人机着陆。0 层模型在起飞过程中产生了显著的错误。

在实验中, 我们观察到没有谱归一化的 $Neural-Lander$ 甚至可能导致控制器输出异常, 进而引发崩溃, 这在经验上表明在训练深度神经网络和设计控制器时使用谱归一化的必要性。

F. Trajectory Tracking Performance

为了展示我们的算法能够处理在物理建模动力学会显著更困难的更复杂环境, 我们设计了一个任务, 即跟踪如图5所示的、非常接近桌面的椭圆轨迹, 其周期为10秒。该轨迹部分位于桌面上, 存在显著的地面效应, 并在桌边有向自由空间的急剧过渡。我们在此测试中比较了 $Neural-Lander$ 和基线控制器的表现。

为了对桌子附近的复杂动态进行建模, 我们手动在靠近桌子的空间飞行无人机以收集另一组数据。我们使用 $x-y$ 位置作为额外输入特征训练了一个新的 ReLU DNN 模型: $\hat{\mathbf{f}}_a(\mathbf{p}, \mathbf{v}, R, \mathbf{u})$ 。与设定点实验类似, 频谱的好处

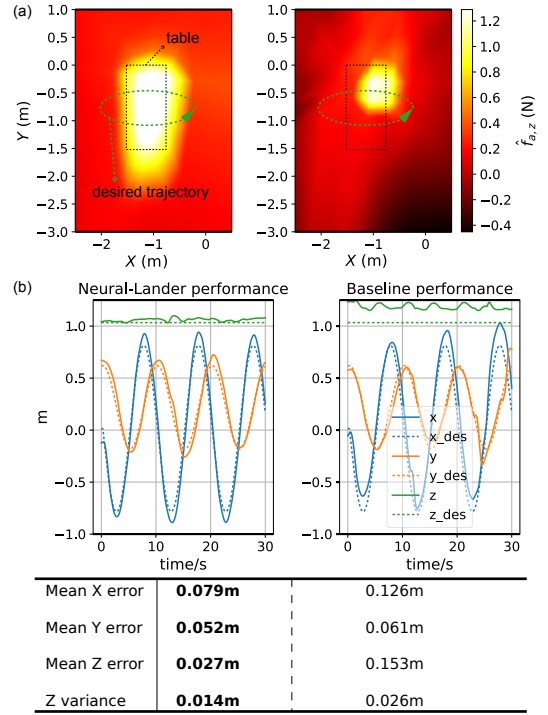


图5: (a) 学习到的 $\hat{f}_{a,z}$ 与 x 和 y 的热图, 其他输入固定。(左) 具有谱归一化的 ReLU 网络。(右) 不具有谱归一化的 ReLU 网络。(b) 跟踪性能和统计数据。

归一化可以在图5(a)中看到, 其中只有频谱归一化的 DNN 显示出清晰的桌子边界。

图5(b)显示, $Neural-Lander$ 在所有 x 、 y 和 z 轴上跟踪期望位置轨迹的表现优于基线控制器。此外, $Neural-Lander$ 在高度方面的方差更低, 即使在桌子边缘, 当无人机飞过桌子时, 控制器也能捕捉到地面效应的变化。

总之, 在多种地面交互场景下的实验结果显示, 由 $Neural-Lander$ 获得的跟踪误差要小得多, $Neural-Lander$ 本质上是具有谱归一化 DNN 前馈抵消的非线性跟踪控制器。

七、结论

在本文中, 我们提出了 $Neural-Lander$, 一种基于深度学习的非线性控制器, 能够在保证稳定性的前提下实现精确的四旋翼降落。与基线控制器相比, $Neural-Lander$ 能显著提高控制性能。主要优点包括: (1) 我们的方法可以从耦合的非稳态气动和飞行器动力学中学习, 比理论地面效应模型提供更准确的估计; (2) 我们的模型能够同时捕捉地面效应和其他非主导气动, 在所有轴上 (x 、 y 和 z) 的表现优于传统控制器; (3) 我们对方法进行了严格的理论分析, 保证了控制器的稳定性, 这也意味着对未见领域的泛化能力。

²Demo videos: <https://youtu.be/FLlsG0S78ik>

Future work includes further generalization of the capabilities of *Neural-Lander* handling unseen state and disturbance domains, such as those generated by a wind fan array.

ACKNOWLEDGEMENT

The authors thank Joel Burdick, Mory Gharib and Daniel Pastor Moreno. The work is funded in part by Caltech's Center for Autonomous Systems and Technologies and Raytheon Company.

REFERENCES

- [1] I. Cheeseman and W. Bennett, "The effect of ground on a helicopter rotor in forward flight," 1955.
- [2] K. Nonaka and H. Sugizaki, "Integral sliding mode altitude control for a small model helicopter with ground effect compensation," in *American Control Conference (ACC)*, 2011. IEEE, 2011, pp. 202–207.
- [3] L. Danjun, Z. Yan, S. Zongying, and L. Geng, "Autonomous landing of quadrotor based on ground effect modelling," in *Control Conference (CCC), 2015 34th Chinese*. IEEE, 2015, pp. 5647–5652.
- [4] F. Berkenkamp, A. P. Schoellig, and A. Krause, "Safe controller optimization for quadrotors with Gaussian processes," in *Proc. of the IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 493–496. [Online]. Available: <https://arxiv.org/abs/1509.01066>
- [5] P. Abbeel, A. Coates, and A. Y. Ng, "Autonomous helicopter aerobatics through apprenticeship learning," *The International Journal of Robotics Research*, vol. 29, no. 13, pp. 1608–1639, 2010.
- [6] A. Punjani and P. Abbeel, "Deep learning helicopter dynamics models," in *Robotics and Automation (ICRA), 2015 IEEE International Conference on*. IEEE, 2015, pp. 3223–3230.
- [7] S. Bansal, A. K. Akametalu, F. J. Jiang, F. Laine, and C. J. Tomlin, "Learning quadrotor dynamics using neural network for flight control," in *Decision and Control (CDC), 2016 IEEE 55th Conference on*. IEEE, 2016, pp. 4653–4660.
- [8] Q. Li, J. Qian, Z. Zhu, X. Bao, M. K. Helwa, and A. P. Schoellig, "Deep neural networks for improved, impromptu trajectory tracking of quadrotors," in *Robotics and Automation (ICRA), 2017 IEEE International Conference on*. IEEE, 2017, pp. 5183–5189.
- [9] S. Zhou, M. K. Helwa, and A. P. Schoellig, "Design of deep neural networks as add-on blocks for improving impromptu trajectory tracking," in *Decision and Control (CDC), 2017 IEEE 56th Annual Conference on*. IEEE, 2017, pp. 5201–5207.
- [10] C. Sánchez-Sánchez and D. Izzo, "Real-time optimal control via deep neural networks: study on landing problems," *Journal of Guidance, Control, and Dynamics*, vol. 41, no. 5, pp. 1122–1135, 2018.
- [11] S. Balakrishnan and R. Weil, "Neurocontrol: A literature survey," *Mathematical and Computer Modelling*, vol. 23, no. 1-2, pp. 101–117, 1996.
- [12] M. T. Frye and R. S. Provence, "Direct inverse control using an artificial neural network for the autonomous hover of a helicopter," in *Systems, Man and Cybernetics (SMC), 2014 IEEE International Conference on*. IEEE, 2014, pp. 4121–4122.
- [13] H. Suprijono and B. Kusumoputro, "Direct inverse control based on neural network for unmanned small helicopter attitude and altitude control," *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)*, vol. 9, no. 2-2, pp. 99–102, 2017.
- [14] F. Berkenkamp, M. Turchetta, A. P. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," in *Proc. of Neural Information Processing Systems (NIPS)*, 2017. [Online]. Available: <https://arxiv.org/abs/1705.08551>
- [15] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," *arXiv preprint arXiv:1802.05957*, 2018.
- [16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.
- [17] T. Salimans and D. P. Kingma, "Weight normalization: A simple reparameterization to accelerate training of deep neural networks," in *Advances in Neural Information Processing Systems*, 2016, pp. 901–909.
- [18] P. L. Bartlett, D. J. Foster, and M. J. Telgarsky, "Spectrally-normalized margin bounds for neural networks," in *Advances in Neural Information Processing Systems*, 2017, pp. 6240–6249.
- [19] J. Slotine and W. Li, *Applied Nonlinear Control*. Prentice Hall, 1991.
- [20] S. Bandyopadhyay, S.-J. Chung, and F. Y. Hadaegh, "Nonlinear attitude control of spacecraft with a large captured object," *Journal of Guidance, Control, and Dynamics*, vol. 39, no. 4, pp. 754–769, 2016.
- [21] X. Shi, K. Kim, S. Rahili, and S.-J. Chung, "Nonlinear control of autonomous flying cars with wings and distributed electric propulsion," in *2018 IEEE Conference on Decision and Control (CDC)*. IEEE, 2018, pp. 5326–5333.
- [22] D. Morgan, G. P. Subramanian, S.-J. Chung, and F. Y. Hadaegh, "Swarm assignment and trajectory optimization using variable-swarm, distributed auction assignment and sequential convex programming," *Int. J. Robotics Research*, vol. 35, no. 10, pp. 1261–1285, 2016.
- [23] H. Khalil, *Nonlinear Systems*, ser. Pearson Education. Prentice Hall, 2002.
- [24] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," *arXiv preprint arXiv:1611.03530*, 2016.
- [25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [26] B. Neyshabur, S. Bhojanapalli, D. McAllester, and N. Srebro, "A pac-bayesian approach to spectrally-normalized margin bounds for neural networks," *arXiv preprint arXiv:1707.09564*, 2017.
- [27] G. K. Dziugaite and D. M. Roy, "Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data," *arXiv preprint arXiv:1703.11008*, 2017.
- [28] B. Neyshabur, S. Bhojanapalli, D. McAllester, and N. Srebro, "Exploring generalization in deep learning," in *Advances in Neural Information Processing Systems*, 2017, pp. 5947–5956.
- [29] S.-J. Chung, S. Bandyopadhyay, I. Chang, and F. Y. Hadaegh, "Phase synchronization control of complex networks of Lagrangian systems on adaptive digraphs," *Automatica*, vol. 49, no. 5, pp. 1148–1161, 2013.
- [30] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," 2017.

未来的工作包括进一步推广 *Neural-Lander* 的能力, 以处理未见过的状态和干扰领域, 例如由风扇阵列产生的情况。

致谢

作者感谢 Joel Burdick、Mory Gharib 和 Daniel Pastor Moreno。本工作部分由加州理工学院自主系统与技术中心和雷神公司资助。

参考文献

- [1] I. Cheeseman 和 W. Bennett, “直升机旋翼在前进飞行中地面效应的影响,” 1955年。[2] K. Nonaka 和 H. Sugizaki, “具有地面效应补偿的小型模型直升机积分滑模高度控制,” 载于 *American Control Conference (ACC)*, 2011。IEEE, 2011, 页码 202–207。[3] L. Dan jun, Z. Yan, S. Zongying 和 L. Geng, “基于地面效应建模的四旋翼自主着陆,” 载于 *Control Conference (CCC)*, 2015 34th Chinese。IEEE, 2015, 页码 5647–5652。[4] F. Berkenkamp, A. P. Schoellig 和 A. Krause, “基于高斯过程的四旋翼安全控制器优化,” 载于 *Proc. of the IEEE International Conference on Robotics and Automation (ICRA)*, 2016, 页码 493–496。[在线]. 可获得: <https://arxiv.org/abs/1509.01066> [5] P. Abbeel, A. Coates 和 A. Y. Ng, “通过师徒学习实现自主直升机特技飞行,” *The International Journal of Robotics Research*, 第29卷, 第13期, 页码 1608–1639, 2010年。[6] A. Punjani 和 P. Abbeel, “深度学习直升机动力学模型,” 载于 *Robotics and Automation (ICRA)*, 2015 *IEEE International Conference on*。IEEE, 2015, 页码 3223–3230。[7] S. Bansal, A. K. Akametalu, F. J. Jiang, F. Laine 和 C. J. Tomlin, “学习 q
- [18] P. L. Bartlett, D. J. Foster, 和 M. J. Telgarsky, “神经网络的谱归一化边界,” 收录于 *Advances in Neural Information Processing Systems*, 2017, 页 6240–6249。[19] J. Slotine 和 W. Li, *Applied Nonlinear Control*. Prentice Hall, 1991。[20] S. Bandyopadhyay, S.-J. Chung, 和 F. Y. Hadaegh, “带有大型捕获物体的航天器非线性姿态控制,” *Journal of Guidance, Control, and Dynamics*, 第 39 卷, 第 4 期, 页 754–769, 2016。[21] X. Shi, K. Kim, S. Rahili, 和 S.-J. Chung, “带翅膀和分布式电推进的自主飞行汽车非线性控制,” 收录于 *2018 IEEE Conference on Decision and Control (CDC)*. IEEE, 2018, 页 5326–5333。[22] D. Morgan, G. P. Subramanian, S.-J. Chung, 和 F. Y. Hadaegh, “使用可变群体、分布式拍卖分配和顺序凸优化的群体分配与轨迹优化,” *Int. J. Robotics Research*, 第 35 卷, 第 10 期, 页 1261–1285, 2016。[23] H. Khalil, *Nonlinear Systems*, 系列 Pearson Education. Prentice Hall, 2002。[24] C. Zhang, S. Bengio, M. Hardt, B. Recht, 和 O. Vinyals, “理解深度学习需要重新思考泛化”